

DX NetOps Portal Dashboard Manager: Deployment & Administration Guide

This document provides a complete, step-by-step operational guide for installing, configuring, and troubleshooting the **NetOps Portal Dashboard Manager** on a fresh installation of Rocky Linux 9.

1. Environment Requirements & Prerequisites

System Requirements

- **Operating System:** Rocky Linux 9.x (x86_64)
- **User Privileges:** Standard user with `sudo` administrative rights
- **Network Access:** Outbound Internet access (for package installation and Git cloning) and network line-of-sight to target DX NetOps Portal instances.

Network Port Allocation

Port	Protocol	Usage	Description
80	TCP	Inbound	Web UI Access (Mapped to Container Port 5000)
443	TCP	Inbound	Web UI Access (Mapped to Container Port 5000 for strict firewall environments)
8181 / 8182	TCP	Outbound	NetOps Portal Web Services API (HTTP/HTTPS)

2. Server Preparation & Docker Installation

Step 1: Update System Packages & Install Dependencies

Log into the Rocky Linux server via SSH and execute:

Bash

None

```
sudo dnf update -y
sudo dnf install -y git dnf-plugins-core
```

Step 2: Install Docker Engine & Docker Compose Plugin

Add the official Docker repository and install the engine components:

Bash

None

```
# Add Docker CE Repository
sudo dnf config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo

# Install Docker CE, CLI, Containerd, and Compose Plugin
sudo dnf install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin

# Enable and start the Docker service
sudo systemctl enable --now docker
```

Step 3: Configure Non-Root User Docker Permissions

Add your local user account to the `docker` group to allow execution of container commands without `sudo`:

Bash

None

```
sudo usermod -aG docker $USER
exec su - $USER
```

3. Host Firewall Configuration (firewalld)

Rocky Linux 9 utilizes `nftables` via `firewalld` by default. To allow traffic into the container across enterprise lab boundaries, open standard HTTP/HTTPS ports and configure interface masquerading.

Bash

None

```
# Open HTTP (80) and HTTPS (443) services
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-service=https --permanent

# Enable IP masquerading for Docker container network forwarding
sudo firewall-cmd --zone=public --add-masquerade --permanent
sudo firewall-cmd --zone=trusted --add-interface=docker0
--permanent

# Reload firewalld to apply changes
sudo firewall-cmd --reload
```

4. Application Deployment

Step 1: Clone the GitHub Repository

Clone the project repository to your desired directory (e.g., `/projects` or home directory):

Bash

None

```
git clone https://github.com/chrisryanbrcm/portal-dash-mgr.git
cd portal-dash-mgr
```

Step 2: Verify `docker-compose.yml` Configuration

Ensure your `docker-compose.yml` reflects the required host-to-container port mappings (binding external ports **80** and **443** to container port **5000**):

YAML

None

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "80:5000"
      - "443:5000"
    restart: always
```

Step 3: Build and Launch the Container

Bash

None

```
# Build the image and start the application in detached mode
docker compose up -d --build
```

5. Post-Deployment Verification

Perform local and external sanity checks to ensure the application stack is healthy.

Local Health Check (CLI)

Run a curl request against the local health endpoint on port 80:

Bash

None

```
curl -I http://localhost/health
```

Expected Output: HTTP/1.1 200 OK

Remote UI Check (Browser)

Open a web browser on your workstation and navigate to either:

- <http://<rocky-linux-ip>>
- <http://<rocky-linux-ip>:443>

6. Troubleshooting & Diagnostics Guide

Diagnostic Tooling Matrix

Test Objective	Command / Tool	Target Result
Container Status	<code>docker ps</code>	Status showing Up on 0.0.0.0:80->5000/tcp, 0.0.0.0:443->5000/tcp
Container Logs	<code>docker compose logs -f</code>	* Running on http://127.0.0.1:5000 (No Python exceptions)
Host Socket Binding	<code>sudo ss -tulpn grep -E '80 443'</code>	docker-proxy process listening on 0.0.0.0:80 and 0.0.0.0:443
Internal Routing	<code>curl -I http://127.0.0.1:5000/health</code>	HTTP/1.1 200 OK
Firewall Status	<code>sudo firewall-cmd --list-all</code>	services: http https, masquerade: yes

Common Issues & Resolution Steps

Issue 1: Web Interface Unreachable from External Workstation

- **Symptoms:** Browser times out when accessing `http://<rocky-ip>`, but `curl http://localhost/health` inside SSH returns `200 OK`.
- **Root Cause:** Enterprise hypervisor, lab security groups, or `firewalld` dropping non-standard high ports (e.g., `5000`).
- **Resolution:**
 1. Verify the app is bound to ports 80 and 443 in `docker-compose.yml`.
 2. Restart `firewalld` and Docker to re-inject rules:
 3. Bash

None

```
sudo firewall-cmd --reload
sudo systemctl restart docker
cd /path/to/portal-dash-mgr && docker compose up -d
```

4.

Issue 2: `firewalld` Reload Clears Docker Forwarding Rules

- **Symptoms:** App works immediately after `docker compose up`, but breaks whenever `firewall-cmd --reload` is run.
- **Root Cause:** On RHEL/Rocky 9, `firewalld` resets `iptables` and `nftables` chains, wiping Docker's dynamic routing rules.
- **Resolution:**

Always restart the Docker daemon after reloading `firewalld`:
- Bash

None

```
sudo firewall-cmd --reload
sudo systemctl restart docker
```

•

Issue 3: REST API `404 Not Found` During Export/Import

- **Symptoms:** Export or Import request returns a status code `404` inside the application UI.
- **Root Cause:**
 1. The target dashboard is an Out-Of-The-Box (OOTB) system dashboard (NetOps Portal API only supports custom dashboards).
 2. Endpoint path mismatch (`/pc/center/rest/...` vs `/pc/center/webservice/...`).
- **Resolution:**
 1. Verify the dashboard being exported was created manually in NetOps Portal.
 2. Verify that `app.py` uses the `/webservice/` path:
 - Export Endpoint: `/pc/center/webservice/dashboards/{page_id}`

- Import Endpoint: `/pc/center/webservice/dashboards/import`

Issue 4: Permission Denied Running Docker Commands

- **Symptoms:** `permission denied while trying to connect to the Docker daemon socket.`
- **Root Cause:** Current user is not in the `docker` system group or group membership has not refreshed in the active shell.
- **Resolution:**
- Bash

None

```
sudo usermod -aG docker $USER  
exec su - $USER
```


7. Ongoing Maintenance & Upgrades

To update the application when new releases or code improvements are pushed to GitHub:

Bash

None

```
cd portal-dash-mgr
```

```
# Pull latest commits from GitHub
```

```
git pull origin main
```

```
# Rebuild and restart the container
```

```
docker compose up -d --build
```